

UNIVERSIDAD DA VINCI DE GUATEMALA
FACULTAD DE INGENIERÍA, INDUSTRIA Y TECNOLOGÍA
CARRERA: LICENCIATURA EN INGENIERÍA EN SISTEMAS
CURSO: ESTRUCTURA DE DATOS



TDA ÁRBOL DE EXPRESIÓN

Evaluación final

Jimmy Javier Martínez Sipac
Carnet 202302745

Guatemala, 26 de Mayo de 2024

Contenido

Introducción	5
TDA Árbol de expresión	6
¿Qué es un árbol binario?	6
Tipos de árboles binarios	7
Árbol binario de búsqueda (BST)	7
Árbol binario de búsqueda equilibrado (AVL)	7
Árbol binario de búsqueda rojo-negro	7
Árbol binario completo	7
Árbol binario perfecto	7
Operaciones.....	7
Inserción	7
Eliminación.....	8
Búsqueda.....	8
Recorrido en orden (in-order traversal)	8
Recorrido en preorden (pre-order traversal)	8
Recorrido en postorden (post-order traversal).....	8
Altura del árbol.....	8
Número de nodos.....	8
¿Qué es la traducción infija a postfija?	9
Operaciones.....	10
Lectura de la expresión infija.....	10
Identificación de operandos y operadores	10
Uso de una pila y una cola	10
Aplicación de reglas de precedencia de operadores	10
Comparación de precedencia de operadores.....	10
Comparación de precedencia de operadores.....	10
Extracción de operadores de la pila	10
Finalización de la traducción	11
Obtención de la expresión postfija	11
Especificación formal del TDA	12
2. Invariante de TDA:.....	12
3. Operaciones.....	12

4. Definición de las operaciones.....	13
CONSTRUCTORA.....	13
OPERACIONES PROPIAS	13
Referencias.....	17
Anexos.....	18
Programa fuente: Clase que implementa en Java el TDA Árbol de expresión	18
Plan de pruebas	21
Conclusiones.....	22

Introducción

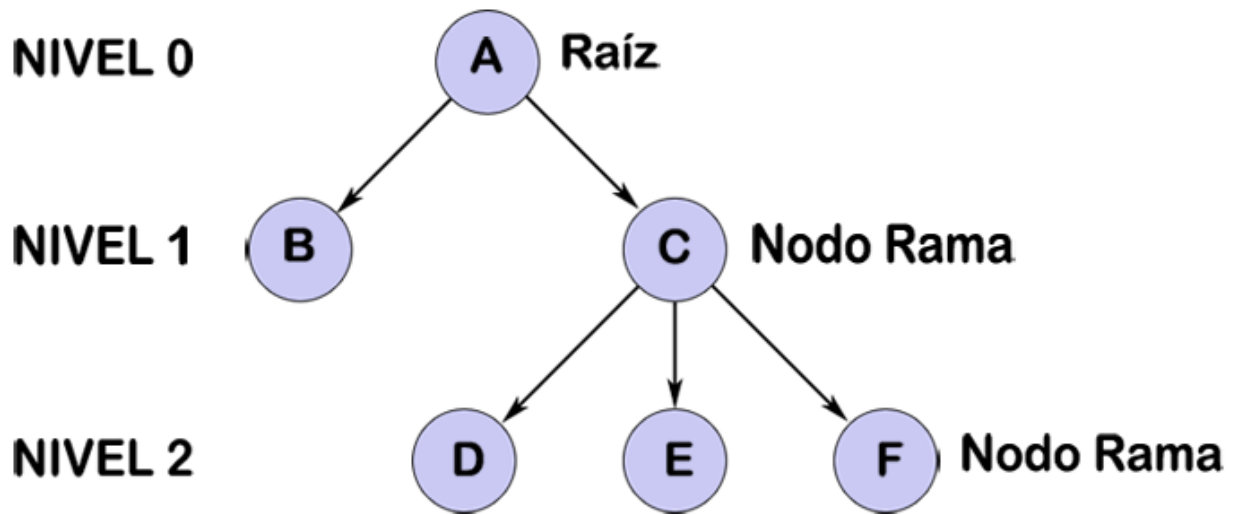
El desarrollo de la evaluación final implica presentar la especificación e implementación del TDA Árbol de expresión. Un Tipo de Dato Abstracto (TDA) se define como un modelo que especifica los valores y las operaciones que se pueden realizar sobre ellos, con la intención de que el usuario no necesite conocer los detalles internos de la representación o la implementación de las operaciones.

En el presente documento, se explorará la implementación del TDA de árboles binarios que aborda la traducción de expresiones infijas a posfijas. Este proceso se realiza a partir de un archivo de texto que contiene una serie de expresiones infijas, una por línea. La tarea implica transformar cada expresión infija en su equivalente posfijo, siguiendo las reglas de precedencia de operadores. La verificación se realiza mediante un archivo que contiene múltiples líneas de expresiones infijas, generando como resultado la cadena que representa cada expresión en forma posfija. Esta implementación, realizada en el lenguaje de programación Java, examina sus componentes clave, operaciones básicas y consideraciones de diseño.

En los anexos, se proporcionan referencias a los repositorios donde se implementó el TDA en lenguaje de código Java, así como un manual de usuario para utilizar la implementación desarrollada.

TDA Árbol de expresión

¿Qué es un árbol binario?



Esquema de árbol binario [Esquema]. (ANIEI)

Un árbol binario es una estructura de datos jerárquica utilizada en informática y matemáticas para representar datos de manera organizada. Se compone de un conjunto de nodos conectados entre sí mediante enlaces llamados "ramas" o "aristas". Cada nodo en un árbol binario puede tener hasta dos nodos hijos, conocidos como el "hijo izquierdo" y el "hijo derecho". Estos nodos hijos, a su vez, pueden tener sus propios hijos, lo que permite una estructura jerárquica ramificada.

En un árbol binario, un nodo especial llamado "raíz" se encuentra en la parte superior y no tiene un nodo padre. Cada nodo en el árbol, excepto la raíz, tiene exactamente un nodo padre. Los nodos que no tienen hijos se llaman "hojas" o "nodos terminales". La profundidad de un árbol binario se refiere a la longitud del camino más largo desde la raíz hasta una hoja, mientras que la altura del árbol es la longitud máxima de cualquier camino desde la raíz hasta una hoja.

Los árboles binarios se utilizan ampliamente en la implementación de algoritmos y estructuras de datos en informática, debido a su capacidad para organizar y manipular datos de manera eficiente. Pueden representar una variedad de estructuras de datos, como árboles de búsqueda binaria, árboles de expresión, y árboles AVL, entre otros. Además, los árboles binarios son fundamentales en la comprensión y optimización de

algoritmos en áreas como la búsqueda, la ordenación y la optimización de bases de datos.

Tipos de árboles binarios

Árbol binario de búsqueda (BST)

En un BST, para cada nodo, todos los nodos en el subárbol izquierdo tienen valores menores que el nodo raíz y todos los nodos en el subárbol derecho tienen valores mayores. Esto facilita la búsqueda eficiente de elementos, ya que se pueden aplicar algoritmos de búsqueda binaria.

Árbol binario de búsqueda equilibrado (AVL)

Un AVL es un tipo de árbol binario de búsqueda en el que se asegura que la diferencia entre las alturas de los subárboles izquierdo y derecho de cualquier nodo sea como máximo 1. Esto garantiza un tiempo de búsqueda eficiente y equilibra la estructura del árbol.

Árbol binario de búsqueda rojo-negro

Similar al AVL, un árbol rojo-negro es un tipo de árbol binario de búsqueda que cumple ciertas restricciones de balanceo. Cada nodo tiene un atributo de color que puede ser rojo o negro, y se aplican reglas específicas de balanceo para mantener la estructura equilibrada.

Árbol binario completo

En un árbol binario completo, cada nivel del árbol está completamente lleno, excepto posiblemente el último nivel, que se llena de izquierda a derecha. Esto garantiza que el árbol esté equilibrado y que todas las hojas estén en el mismo nivel.

Árbol binario perfecto

Un árbol binario perfecto es un tipo de árbol binario completo en el que todos los niveles están completamente llenos. Esto significa que el número de nodos en cada nivel es el máximo posible para ese nivel, lo que resulta en un árbol perfectamente equilibrado.

Operaciones

Con los árboles binarios se pueden realizar las siguientes operaciones:

Insertión

Agregar un nuevo nodo al árbol binario, manteniendo la propiedad de ordenación

Eliminación

Eliminar un nodo específico del árbol, manteniendo la estructura y las propiedades del árbol.

Búsqueda

Buscar un nodo con un valor específico en el árbol. Dependiendo del tipo de árbol binario, la búsqueda puede ser más eficiente.

Recorrido en orden (in-order traversal)

Visitar todos los nodos del árbol en un orden específico. En un árbol binario de búsqueda, el recorrido en orden visita primero el subárbol izquierdo, luego el nodo actual y finalmente el subárbol derecho.

Recorrido en preorden (pre-order traversal)

Visitar todos los nodos del árbol en un orden específico. En un recorrido preorden, el nodo actual se visita antes de visitar sus subárboles izquierdo y derecho.

Recorrido en postorden (post-order traversal)

Visitar todos los nodos del árbol en un orden específico. En un recorrido postorden, los subárboles izquierdo y derecho se visitan antes de visitar el nodo actual.

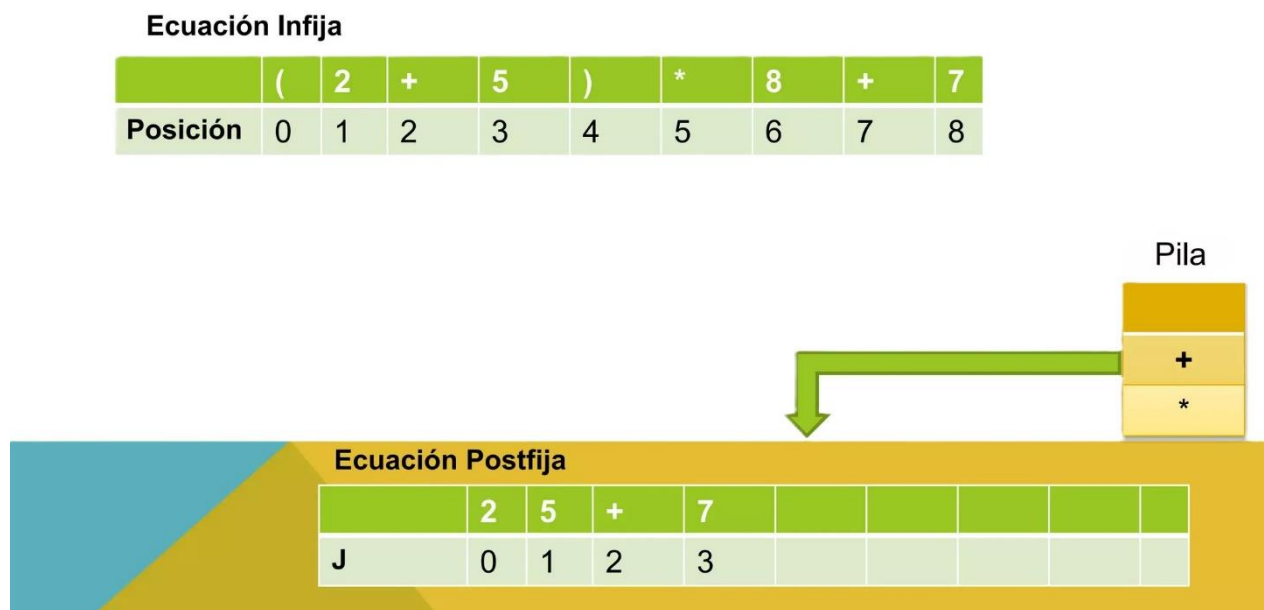
Altura del árbol

Calcular la altura del árbol, es decir, la longitud del camino más largo desde la raíz hasta una hoja.

Número de nodos

Contar el número total de nodos en el árbol.

¿Qué es la traducción infija a postfija?



Esquema de traducción infija a postfija [Esquema]. (Eddy Antonio Scotto Castillo, 2015)

La traducción de una expresión infija a posfija implica convertir una expresión matemática escrita en la forma tradicional, donde los operadores se colocan entre los operandos, a una forma postfix, donde los operadores se colocan después de sus operandos correspondientes. Este proceso se lleva a cabo mediante el uso de reglas específicas para determinar el orden de los operadores y garantizar la correcta evaluación de la expresión.

Durante la traducción, se recorre la expresión infija de izquierda a derecha y se utilizan una pila y una cola para almacenar temporalmente los operadores y operandos, respectivamente. Se utilizan reglas de precedencia de operadores para determinar cuándo se deben realizar las operaciones y en qué orden se deben colocar los operadores en la expresión posfija resultante.

Cada vez que se encuentra un operando, se agrega a la cola de salida. Cuando se encuentra un operador, se compara con el operador en la parte superior de la pila. Si el operador actual tiene una precedencia menor o igual que el operador en la parte superior de la pila, se saca el operador de la pila y se agrega a la cola de salida. Este proceso se repite hasta que se encuentra un operador con menor precedencia que el operador actual, o hasta que la pila esté vacía. Luego, el operador actual se coloca en la pila.

Una vez que se ha recorrido toda la expresión infija, cualquier operador restante en la pila se saca y se agrega a la cola de salida. La cola de salida resultante contiene la

expresión en forma posfija, lista para ser evaluada de manera eficiente utilizando una pila u otro método adecuado.

Operaciones

Durante la traducción de infija a postfija se pueden realizar las siguientes operaciones:

Lectura de la expresión infija

Se lee la expresión matemática en formato infijo, que puede contener operandos, operadores y paréntesis.

Identificación de operandos y operadores

Se identifican los operandos (números) y los operadores (+, -, *, /, etc.) en la expresión infija.

Uso de una pila y una cola

Se utilizan estructuras de datos como una pila y una cola para almacenar temporalmente los operadores y operandos, respectivamente.

Aplicación de reglas de precedencia de operadores

Se aplican reglas de precedencia de operadores para determinar el orden en el que deben colocarse los operadores en la expresión posfija.

Comparación de precedencia de operadores

Comparación de precedencia de operadores.

Comparación de precedencia de operadores

Se colocan los operandos directamente en la cola de salida. Los operadores se colocan en la pila de operadores, siguiendo reglas específicas de precedencia.

Extracción de operadores de la pila

Cuando se encuentra un operador con menor precedencia que el operador actual, se extraen los operadores de la pila y se colocan en la cola de salida.

Finalización de la traducción

Una vez que se ha recorrido toda la expresión infija, se vacía la pila de operadores y se colocan en la cola de salida.

Obtención de la expresión posfija

La cola de salida resultante contiene la expresión en formato posfijo, lista para ser evaluada o procesada de manera eficiente.

Especificación formal del TDA

Nombre: Árbol de expresiones

1. Descripción del TDA:

El TDA (Tipo de Dato Abstracto) Árbol de Expresión se encarga de representar y evaluar expresiones matemáticas en formato infijo y posfijo. Su funcionamiento se basa en la lectura de expresiones infijas desde un archivo de texto, su conversión a formato posfijo y su posterior evaluación mediante la construcción de un árbol de expresión.

2. Invariante de TDA:

Para que el TDA Cola Medieval sea válido, debe cumplir con las siguientes invariantes:

2.1 Estructura correcta del árbol

Cada nodo tiene cero o dos hijos, respetando la estructura de un árbol binario.

2.2 Precedencia de operadores preservada

La conversión infija a posfija mantiene la precedencia de los operadores de manera adecuada.

2.3 Evaluación precisa

La evaluación de la expresión posfija produce resultados coherentes y correctos.

2.4 Manejo adecuado de errores

El TDA maneja correctamente cualquier error o excepción durante la lectura, conversión y evaluación de las expresiones.

3. Operaciones

Evaluar
EsOperador
Factorial
ConvertirInfijaAPosfija
LeerLineas
EsVacía
EvaluarNodo

4. Definición de las operaciones

CONSTRUCTORA

4.1 Constructor

4.1.1 Descripción

Permite instanciar una árbol de expresión, recibe como parámetro la expresión postfija. Utiliza una pila para construir el árbol de manera iterativa, procesando cada token de la expresión posfija y creando los nodos correspondientes

4.1.2 Descripción operacional

Itera sobre cada token de la expresión posfija, si el token es un operador, crea un nodo correspondiente y lo agrega a la pila, junto con sus nodos hijos si corresponde. De lo contrario si el token es un operando, crea un nodo correspondiente y lo agrega a la pila. Al finalizar el recorrido de la expresión posfija, verifica que la pila contenga exactamente un nodo. Si es así, asigna ese nodo como la raíz del árbol

4.1.3 Precondición

La expresión posfija proporcionada debe estar correctamente formada y ser válida para ser evaluada como una expresión matemática.

4.1.4 Postcondición

La raíz del árbol contiene el resultado de la expresión posfija evaluada.

OPERACIONES PROPIAS

4.2 Evaluar

4.2.1 Descripción

Este método se encarga de evaluar la expresión matemática representada por el árbol de expresión.

4.2.2 Descripción operacional

Verifica si el nodo es nulo o un operando. Si es un operando, retorna su valor convertido a un número double. Si es un operador, recursivamente evalúa sus nodos hijos y aplica la operación correspondiente. Retorna el resultado de la evaluación del nodo.

4.3.3 Precondición

El nodo proporcionado debe ser válido y pertenecer al árbol de expresión.

4.3.4 Postcondición

Si la precondición se cumple, el método retorna el resultado de la evaluación del nodo dado.

4.3 EsOperador

4.3.1 Descripción

Este método verifica si un token dado es un operador matemático

4.3.2 Descripción operacional

Compara el token con los operadores conocidos: +, -, *, /, !. Retorna verdadero si el token coincide con algún operador, falso en caso contrario.

4.3.3 Precondición

El token proporcionado debe ser una cadena de texto válida.

4.3.4 Postcondición

Si la precondición se cumple, el método retorna verdadero o falso según corresponda

4.4 Factorial

4.4.1 Descripción

Este método calcula el factorial de un número entero no negativo.

4.4.2 Descripción operacional

Calcula el factorial del número dado.

4.4.3 Precondición

El número proporcionado debe ser un entero no negativo.

4.4.4 Postcondición

Si la precondición se cumple, el método retorna el factorial del número.

4.5 ConvertirInfijaAPostfija

4.5.1 Descripción

Este método convierte una expresión matemática en formato infijo a su equivalente en formato posfijo.

4.5.2 Descripción operacional

Convierte una expresión infija en una expresión posfija.

4.4.3 Precondición

La expresión infija proporcionada debe ser una cadena de texto válida.

4.4.4 Postcondición

Si la precondición se cumple, el método retorna la expresión en formato posfijo.

4.6 LeerLineas

4.6.1 Descripción

Este método lee las líneas de un archivo de texto y las almacena en una lista.

4.6.2 Descripción operacional

Lee las líneas de un archivo de texto y las almacena en una lista.

4.6.3 Precondición

El nombre del archivo proporcionado debe ser una cadena de texto válida y el archivo debe existir y ser accesible.

4.6.4 Postcondición

Si la precondición se cumple, el método retorna una lista con las líneas leídas del archivo

4.7 EsVacia

4.7.1 Descripción

Este método verifica si una pila está vacía.

4.7.2 Descripción operacional

Verifica si la pila está vacía.

4.7.3 Precondición

Ninguna.

4.7.4 Postcondición

Si la pila está vacía, el método retorna verdadero, de lo contrario retorna falso.

4.8 EvaluarNodo

4.8.1 Descripción

Este método evalúa recursivamente un nodo del árbol de expresión.

4.8.2 Descripción operacional

Evalúa recursivamente un nodo del árbol de expresión.

4.8.3 Precondición

El nodo proporcionado debe ser válido y pertenecer al árbol de expresión.

4.8.4 Postcondición

Si la precondición se cumple, el método retorna el resultado de la evaluación del nodo.

Referencias

Algoritmos de traduccion de expresiones. (s. f.).

<https://www.infor.uva.es/~cvaca/asigs/AlgInfPost.htm>

Conversion de infija a postfija. (2015, 23 junio). [Diapositivas]. SlideShare.

<https://www.slideshare.net/scotto4/conversion-de-infija-a-postfija>

Guerrero, D. (2012, 10 mayo). *Arboles binario*. Monografias.com.

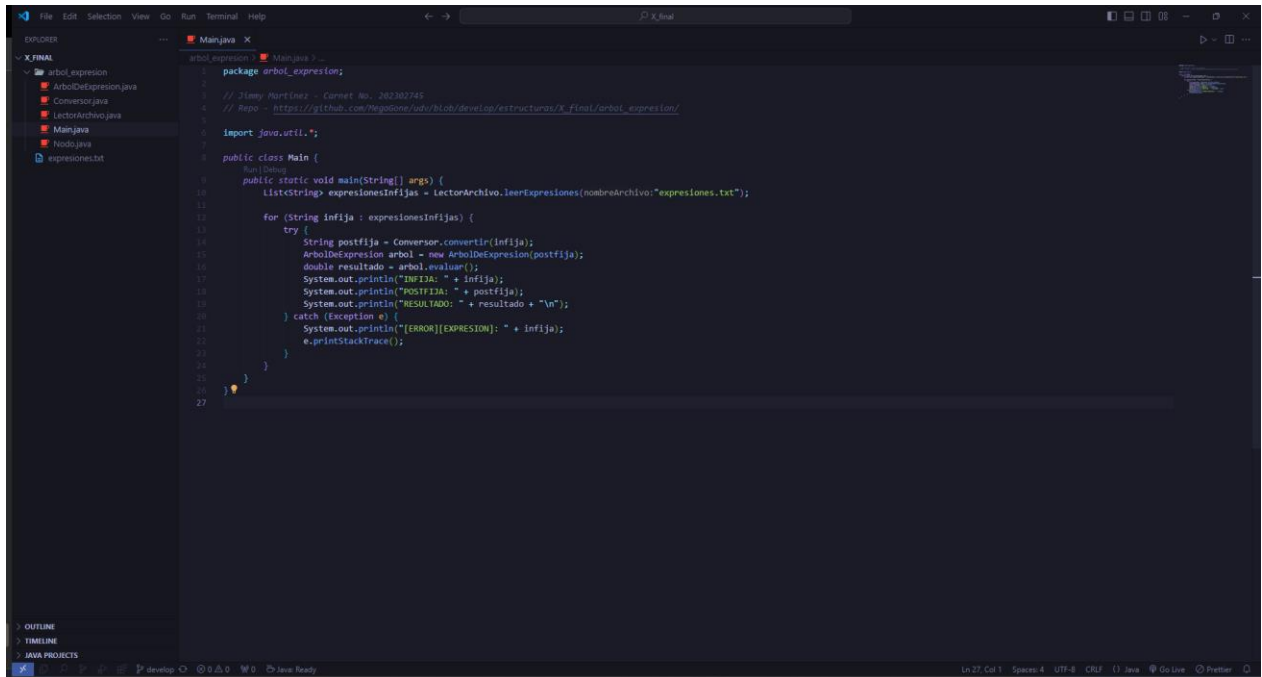
<https://www.monografias.com/trabajos92/arboles-binario/arboles-binario>

LUDA UAM-Azc. (s. f.). http://aniei.org.mx/paginas/uam/CursoPoo/curso_poo_12.html

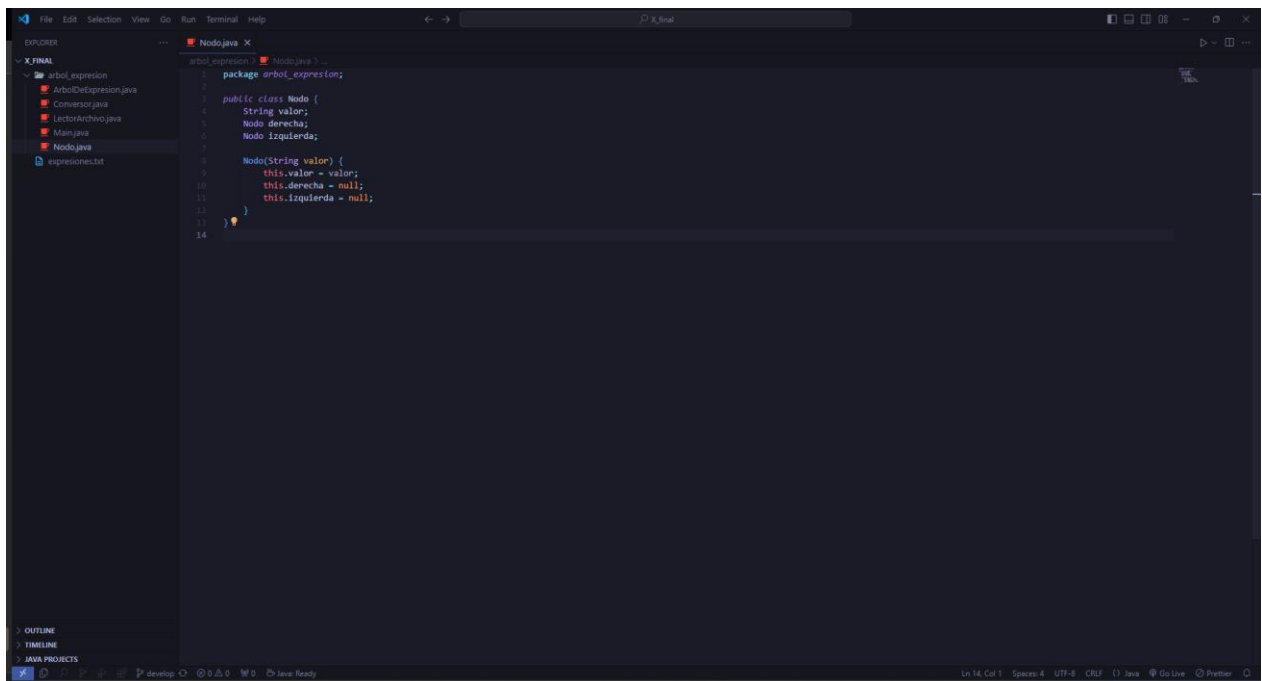
Anexos

Programa fuente: Clase que implementa en Java el TDA Árbol de expresión

https://github.com/MegaGone/udv/blob/develop/estructuras/X_final/arbol_expression/



```
1 package arbol_expression;
2
3 // Jimmy Martinez - Correo: No_202302745
4 // Repo -> https://github.com/MegaGone/udv/blob/develop/estructuras/X_final/arbol_expression/
5
6 import java.util.*;
7
8 public class Main {
9     // Run (Debug)
10     public static void main(String[] args) {
11         List<String> expresionesInfijas = LectorArchivo.leeExpresiones(nombreArchivo:"expresiones.txt");
12
13         for (String infija : expresionesInfijas) {
14             try {
15                 String postfija = Conversor.convertir(infija);
16                 ArbolDeExpresion arbol = new ArbolDeExpresion(postfija);
17                 double resultado = arbol.evaluar();
18                 System.out.println("INFIXA: " + infija);
19                 System.out.println("POSTFIXA: " + postfija);
20                 System.out.println("RESULTADO: " + resultado + "\n");
21             } catch (Exception e) {
22                 System.out.println("[ERROR][EXPRESSION]: " + infija);
23                 e.printStackTrace();
24             }
25         }
26     }
27 }
```



```
1 package arbol_expression;
2
3 public class Nodo {
4     String valor;
5     Nodo derecha;
6     Nodo izquierda;
7
8     Nodo(String valor) {
9         this.valor = valor;
10        this.derecha = null;
11        this.izquierda = null;
12    }
13 }
14
```

```
1 package arbol_expression;
2
3 import java.io.*;
4 import java.util.*;
5
6 public class LectorArchivo {
7     public static List<String> leerExpresiones(String nombreArchivo) {
8         List<String> expresionesInfijas = new ArrayList<>();
9         try (BufferedReader br = new BufferedReader(new FileReader(nombreArchivo))) {
10             String linea;
11             while ((linea = br.readLine()) != null) {
12                 expresionesInfijas.add(linea);
13             }
14         } catch (IOException e) {
15             e.printStackTrace();
16         }
17         return expresionesInfijas;
18     }
19 }
```

```
1 package arbol_expression;
2
3 import java.util.*;
4
5 public class Conversor {
6     public static String convertir(String infijo) {
7         StringBuilder postfijo = new StringBuilder();
8         Stack<Character> pila = new Stack<>();
9         boolean anteriorFueOperador = true; // MANEJAR NÚMEROS NEGATIVOS
10
11         for (char ch : infijo.toCharArray()) {
12             if (Character.isDigit(ch) || ch == '-') {
13                 postfijo.append(ch);
14                 anteriorFueOperador = false;
15             } else if (ch == '(') {
16                 pila.push(ch);
17                 anteriorFueOperador = true;
18             } else if (ch == ')') {
19                 while (!pila.isEmpty() && pila.peek() != '(') {
20                     postfijo.append(c: ' ').append(pila.pop());
21                 }
22                 pila.pop();
23                 anteriorFueOperador = false;
24             } else if (esOperador(ch)) {
25                 if (anteriorFueOperador && ch == '-') {
26                     postfijo.append(ch); // MANEJAR SIGNO NEGATIVO -
27                 } else {
28                     postfijo.append(c: ' ');
29                     while (!pila.isEmpty() && precedencia(ch) <= precedencia(pila.peek())) {
30                         postfijo.append(pila.pop());
31                     }
32                     pila.push(ch);
33                     anteriorFueOperador = true;
34                 }
35             }
36         }
37
38         while (!pila.isEmpty()) {
39             postfijo.append(c: ' ').append(pila.pop());
40         }
41
42         return postfijo.toString().trim();
43     }
44
45     private static boolean esOperador(char ch) {
46         return ch == '+' || ch == '-' || ch == '*' || ch == '/' || ch == '!';
47     }
48 }
```

```
File Edit Selection View Go Run Terminal Help
ArbolDeExpresion.java X

EXPLORER
X_FINAL
  arbol_expresion
    ArbolDeExpresion.java
    Conversor.java
    LecturaArchivo.java
    Main.java
    Nodo.java
    expresiones.txt

ArbolDeExpresion.java
1 package arbol_expresion;
2
3 import java.util.Stack;
4
5 public class ArbolDeExpresion {
6     private Nodo raiz;
7
8     public ArbolDeExpresion(String postfijo) {
9         Stack<Nodo> pila = new Stack<>();
10        for (String token : postfijo.split(regex:"\\s+")) {
11            if (!esOperador(token)) {
12                Nodo nodo = new Nodo(token);
13                if (token.equals(anObject("!")) {
14                    if (pila.isEmpty()) {
15                        throw new IllegalArgumentException(s:"Expresión inválida: no hay suficientes operandos para '!");
16                    }
17                    nodo.derecha = pila.pop();
18                } else {
19                    if (pila.isEmpty()) {
20                        throw new IllegalArgumentException(
21                            "Expresión inválida: no hay suficientes operandos para '" + token + "'");
22                    }
23                    nodo.derecha = pila.pop();
24                    if (pila.isEmpty()) {
25                        throw new IllegalArgumentException(
26                            "Expresión inválida: no hay suficientes operandos para '" + token + "'");
27                    }
28                    nodo.izquierda = pila.pop();
29                }
30                pila.push(nodo);
31            } else {
32                pila.push(new Nodo(token));
33            }
34        }
35
36        if (pila.size() != 1) {
37            return;
38        }
39
40        raiz = pila.pop();
41    }
42
43    public double evaluar() {
44        return evaluar(raiz);
45    }
46
47    private double evaluar(Nodo nodo) {
48        if (nodo == null) {
49            return 0;
50        }
51    }
52}
```

Plan de pruebas

No	Expresión infija	Expresión postfija	Resultado esperado	Resultado obtenido	Resultado final
1	$(-2.9 + (2.7 / 3.0)) * ((5.5 - 2.5)!)$	$2.9\ 2.7\ 3.0\ /\ +\ 5.5\ 2.5\ -\ !\ *$	-12	-12.00	Exitoso
2	$(1+2)*(3/4)$	$1\ 2\ +\ 3\ 4\ /\ *$	2.25	2.25	Exitoso
3	$5 + ((1+2)*4) - 3$	$5\ 1\ 2\ +\ 4\ *\ +\ 3\ -$	14.00	14.00	Exitoso
4	$7 * (2+3) / (4-1)$	$7\ 2\ 3\ +\ *\ 4\ 1\ -\ /\$	11.67	11.67	Exitoso
5	$(8/2)*(2+2)$	$8\ 2\ /\ 2\ 2\ +\ *\$	16.00	16.00	Exitoso

```

package arbol_expression;

// JIany Martinez - Carnet No. 302302745
// Repo - https://github.com/MegaDane/udo/blob/develop/estructuras/X_Final/arbol_expression/

import java.util.*;

public class Main {
    Run(Debug)

    public static void main(String[] args) {

INFIXA: (-2.9+(2.7/3.0))*((5.5-2.5)!)
POSTFIXA: -2.9 2.7 3.0 / + 5.5 2.5 - ! *
RESULTADO: -12.0

INFIXA: (3+4)*(5-6)
POSTFIXA: 3 4 + 5 6 - *
RESULTADO: -7.0

INFIXA: (1+2)*(3/4)
POSTFIXA: 1 2 + 3 4 / *
RESULTADO: 2.25

INFIXA: 5+((1+2)*4)-3
POSTFIXA: 5 1 2 + 4 * + 3 -
RESULTADO: 14.0

INFIXA: (1+2)*(3/4)
POSTFIXA: 1 2 + 3 4 / *
RESULTADO: 2.25

PS C:\repos\udo\estructuras\X_Final>
  
```

Conclusiones

- **Abstracción y encapsulamiento:** Los TDA proporcionan una forma de abstracción de datos al ocultar los detalles de implementación y exponer solo las operaciones relevantes para interactuar con esos datos. Esto promueve el encapsulamiento, lo que significa que los detalles internos de cómo se almacenan y manipulan los datos están ocultos, lo que facilita su uso y mantenimiento.
- **Reutilización y modularidad:** Los TDA fomentan la reutilización de código al proporcionar una interfaz clara y consistente para interactuar con los datos. Esto permite que los desarrolladores utilicen y combinen diferentes TDA en sus programas de manera modular, facilitando la construcción de sistemas complejos a partir de componentes más simples y bien definidos.
- **Flexibilidad y extensibilidad:** Los TDA son flexibles y extensibles, lo que significa que pueden adaptarse y ampliarse para satisfacer diferentes necesidades y requisitos. Los diseñadores pueden crear nuevos TDA o extender TDA existentes para manejar situaciones específicas o agregar funcionalidades adicionales según sea necesario, lo que permite una mayor versatilidad en el diseño y la implementación de sistemas de software.